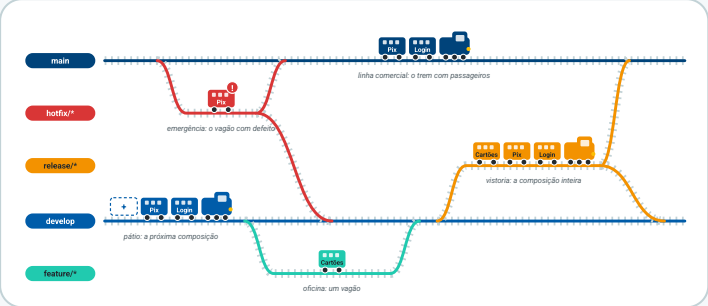
Nos trilhos do GitFlow, numa página

Parte 1 · O trem: para que servem as branches

O **trem** é o software, e cada **vagão**, uma funcionalidade. O trem não pode parar: o trabalho é feito em outros **trilhos** (as branches), e o que fica pronto é engatado de volta.



Trilho	Na ferrovia	O que roda	Sai de → vai para
`main`	a linha comercial	o trem completo, com passageiros	permanente e protegida
`develop`	o pátio de montagem	a próxima composição	permanente e protegida
`feature/*`	a oficina	um vagão, novo ou em melhoria	develop → develop
`release/*`	a vistoria	a composição inteira; só conserto	develop → main e develop
`hotfix/*`	a emergência	um vagão: o que deu defeito	main → main e develop

Parte 2 · O contrato: o que fica registrado e como se junta

Cada **commit** é uma versão rubricada; cada **branch**, uma cópia de trabalho; cada **merge**, a cópia incorporada. Os quatro merges levam o mesmo conteúdo; muda a história que fica.

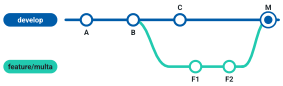
Fast-forward

a minuta não andou: a cópia vira a minuta



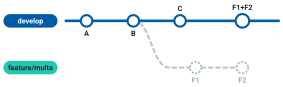
Merge commit

a junção registrada: guarda a cópia



Squash

passar a limpo: uma versão por entrega



Rebase

refazer sobre a minuta atual: versões novas



Conflito é quando duas áreas mudaram a mesma cláusula: uma pessoa decide, e muitas vezes a decisão é de negócio. **Merge back**: tudo o que entra na main volta para a develop.

O número da versão · v2.4.1.121

2 · MAJOR — muda o que já valia

4 · MINOR — novidade

1 · PATCH — correção

121 · BUILD — a montagem que foi para produção

Perguntas para fazer ao time

Já está na main ou só na develop? · Entra nesta release ou na próxima? · É hotfix ou pode esperar? · Qual é a versão em produção? · Fizeram o merge back? · Há quanto tempo essa branch está aberta?

Para levar

1 Cada trabalho no seu trilho: a main é o trem com passageiros, a develop é a próxima composição, os desvios são temporários.

2 Juntar é escolher a história: fast-forward, merge commit, squash ou rebase.

3 Tudo o que chega à main volta para a develop: sem merge back, a correção se perde.